

halfedge：邻域遍历迭代器模块设计文档

src/traversal.rs

halfedge 库

2026-07-06

目录

1	模块定位	2
1.1	内存复用变体：*_into 函数	3
2	核心设计：构造期收集，迭代期不持有借用	3
3	延迟迭代器 (lazy)：零堆分配替代方案	4
3.1	设计动机与权衡	4
3.2	lazy 顶点环算法：CW 探测 + CCW 迭代	4
3.3	lazy 面边界算法	5
3.4	FusedIterator 语义	5
3.5	基准测试结果	5
4	顶点环绕：CCW 旋转规则	6
4.1	两条旋转规则的推导	6
4.2	算法：单三角面片实例	7
5	算法流程图	7
5.1	顶点 outgoing 半边收集	7
5.2	面边界半边收集	8
6	边界判定	8
6.1	边界边	8
6.2	边界顶点	8
7	复杂度	9
8	健壮性	9
9	顶点邻接无向边：VertexAdjacentEdges	9
9.1	构造与算法	10
9.2	与 EdgeIter 的关系	10
9.3	复杂度	10

10 统计信息与诊断	10
10.1 迭代器统计接口	10
10.2 MeshStorage 诊断方法	11
10.3 基准测试: count_hint vs count	11
11 单元测试覆盖	12
12 设计取舍	14

1 模块定位

traversal.rs 在 storage.rs 之上叠加只读拓扑遍历能力, 提供 5 个预收集迭代器 + 6 个 lazy 迭代器 + 无向边/边界环迭代器 + k-ring 邻域 + 边界判定函数, 覆盖半边网格最常见的邻域查询场景:

类别	API	语义
顶点迭代器	VertexRing	v 的所有 outgoing 半边 (CCW)
	VertexAdjacentVerts	v 的所有邻居顶点 (CCW)
	VertexAdjacentFaces	v 周围的所有面 (CCW, 跳过 None)
面迭代器	FaceHalfEdges	面边界环上的所有半边 (next 顺序)
	FaceVertices	面的所有顶点 (每条半边的 tip)
无向边 / 边界环	EdgeIter / MeshStorage::edge_ids	遍历所有唯一无向边
	VertexAdjacentEdges	与顶点 v 关联的所有唯一无向边
	BoundaryLoop / BoundaryLoopLazy	遍历一个边界环 (eager + lazy)
k-ring 邻域	vertex_two_ring(v)	距离正好为 2 的顶点
	vertex_k_ring(v , k)	距离正好为 k 的顶点 (BFS)
	vertex_k_disk(v , k)	距离 $\leq k$ 的所有顶点
工具函数	is_boundary_edge(he)	半边所在边是否为边界边
	is_boundary_vertex(v)	顶点是否位于网格边界
	is_closed(mesh)	是否为闭合网格 (无边边界)
	boundary_loops(mesh)	枚举所有边界环, 返回 Vec<Vec<HalfEdgeId>>
通用遍历原语	collect_outgoing_halfedges(v)	收集 v 的 outgoing 半边 (CCW, Vec)
	collect_face_halfedges(f)	收集面 f 边界环半边 (next 顺序, Vec)
	collect_outgoing_halfedges_into(v , buf)	同上, 填充调用方 buf (内存复用)
	collect_face_halfedges_into(f , buf)	同上, 填充调用方 buf (内存复用)

以上 5 个迭代器均有延迟版本 (后缀 Lazy), 通过 ::lazy() 构造函数获取:

预收集版本	延迟版本
VertexRing::new(&mesh, v)	VertexRing::lazy(&mesh, v)
VertexAdjacentVerts::new(&mesh, v)	VertexAdjacentVerts::lazy(&mesh, v)
VertexAdjacentFaces::new(&mesh, v)	VertexAdjacentFaces::lazy(&mesh, v)
FaceHalfEdges::new(&mesh, f)	FaceHalfEdges::lazy(&mesh, f)
FaceVertices::new(&mesh, f)	FaceVertices::lazy(&mesh, f)

通用遍历原语 (pub fn): 模块还导出两对底层收集函数, 作为下游构建自定义迭代器或分析工具的基础原语:

- `collect_outgoing_halfedges(&mesh, v)`: 返回 v 的所有 outgoing 半边 (CCW 顺序), 已处理开链 / 闭合环 / 孤立顶点等所有边界情况;
- `collect_face_halfedges(&mesh, f)`: 返回面 f 边界环上的所有半边 (next 顺序), 已处理拓扑断开等异常情况。

二者内部实现均使用 `mesh.halfedge_count()` 作为循环上界兜底, 保证在拓扑不一致时也能终止。下游可直接复用这些经过充分测试的收集逻辑, 而不必自己实现 CW+CCW 双向探测。

1.1 内存复用变体: *_into 函数

两个原语各有**内存复用变体** (后缀 `_into`), 签名如下:

- `collect_outgoing_halfedges_into(&mesh, v, buf: &mut Vec<HalfEdgeId>)`
- `collect_face_halfedges_into(&mesh, f, buf: &mut Vec<HalfEdgeId>)`

语义: `buf.clear()` 后填充, **不释放已分配容量**。返回 `Vec` 的版本内部即声明空 `buf`、调用 `_into` 变体填充、最后返回 `buf`, 二者产出完全一致。

适用场景: 热循环中反复遍历不同顶点 / 面的邻域, 例如:

```
1 let mut buf = Vec::new();
2 for v in mesh.vertex_ids() {
3     collect_outgoing_halfedges_into(mesh, v, &mut buf);
4     //     buf
5 }
```

与 lazy 的对比: 两者都降低分配开销, 但侧重点不同——`*_into` 适合需要随机访问或迭代期 `&mut mesh` 的场景 (保留 `eager` 的解耦特性), `lazy` 适合纯只读遍历 (零堆分配, 但持有借用)。

维度	*_into (内存复用)	Lazy (延迟)
堆分配	首次分配, 后续复用容量	零堆分配
迭代期借用	无 (已释放)	持有 <code>&MeshStorage</code>
迭代期 <code>&mut mesh</code>	✓	×
随机访问	✓ (<code>buf[i]</code>)	×

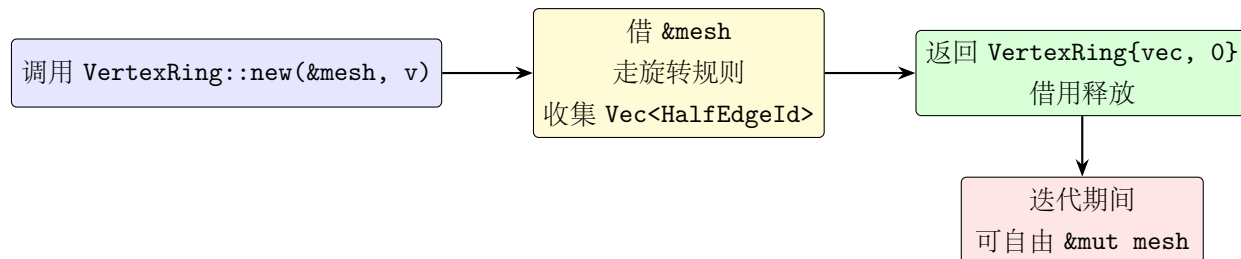
2 核心设计: 构造期收集, 迭代期不持有借用

Rust 的借用检查要求「同一时间要么有多个不可变借用, 要么仅一个可变借用」。若迭代器在生命周期内持有 `&mesh`, 那么形如

```
for he in VertexRing::new(&mesh, v) { ... }
```

的循环体内就无法再写 `mesh.get_halfedge_mut(...)` ——这会显著限制上层使用场景（边遍历边修改拓扑）。

解法：所有迭代器的构造函数签名都是 `fn new(&MeshStorage, Id) -> Self`，内部一次性把要遍历的 ID 收集进 `Vec<Id>`；构造函数返回后，对 `mesh` 的不可变借用立即释放。迭代器结构体只保存 `(Vec<Id>, usize)`，与 `mesh` 完全解耦。



3 延迟迭代器 (lazy)：零堆分配替代方案

3.1 设计动机与权衡

预收集版本 (`::new`) 在构造期分配 `Vec<Id>`，迭代期不持有借用——适合需要随机访问或迭代期修改网格的场景。但在大规模只读遍历中，每个迭代器的 `Vec` 分配成为不必要的开销。

延迟版本 (`::lazy`) 在迭代期持有 `&MeshStorage` 借用，逐跳查询 `next/prev/twin`，零堆分配。两种版本的权衡对比：

维度	预收集 <code>::new</code>	延迟 <code>::lazy</code>
构造期时间	$O(d)$ （走 CW+CCW 收集）	$O(d)$ （仅 CW 探测尽头）
构造期空间	$O(d)$ (<code>Vec<Id></code>)	$O(1)$ （仅记录起点/终点）
迭代期时间	$O(1)$ / <code>next</code> （数组下标）	$O(1)$ / <code>next</code> （指针追逐）
迭代期借用	无（已释放）	持有 <code>&MeshStorage</code>
迭代期可 <code>&mut mesh</code>	✓	×
<code>FusedIterator</code>	✓	✓
产出顺序一致性	—	与预收集版本完全一致

3.2 lazy 顶点环算法：CW 探测 + CCW 迭代

预收集版本从 `start` 出发，先走 CCW 收集、再走 CW 补全、最后拼接。lazy 版本不能用 `Vec` 存储，因此采用两遍策略：

第 1 遍（构造期）——沿 CW 方向探测开链尽头：

$$\text{cw_next}(h) = h.\text{twin}.\text{next}$$

从 `start` 出发，反复应用 `cw_next`，直到：

- 回到 `start` $\Rightarrow$ 闭合环，记录 `closed = true`；

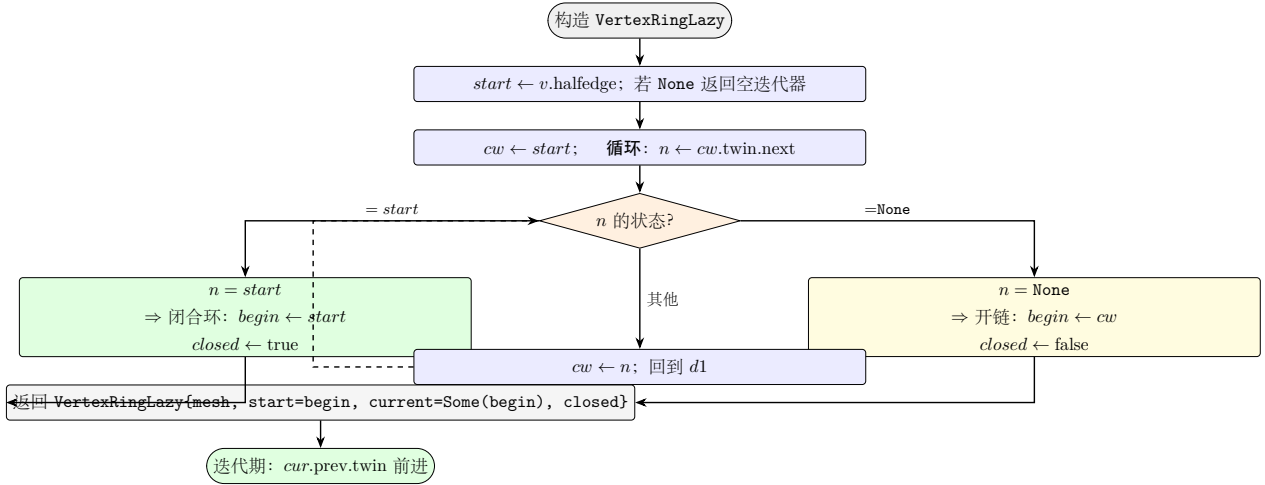
- 撞到 `None` $\Rightarrow$ 开链，记录尽头 `cw_end`。

第 2 遍（迭代期）——沿 CCW 方向逐个产出：

$$ccw_next(h) = h.prev.twin$$

- 闭合环：从原始 `start` 开始迭代，回到 `start` 时终止；
- 开链：从 `cw_end` 开始迭代，撞到 `None` 时终止。

起点一致性证明：预收集版本的产出顺序为 [CW 逆序, `start`, CCW]。对开链，CW 逆序的最后一个元素正是 CW 方向的尽头 `cw_end`，因此 `lazy` 从 `cw_end` 开始沿 CCW 产出，顺序与预收集完全一致。对闭合环，两者都从 `start` 开始。



3.3 lazy 面边界算法

面边界环始终是闭合的（长度 = 面边数），因此 `lazy` 版本更简单：从 `f.halfedge` 出发，沿 `next` 逐跳前进，回到起点时终止。

3.4 FusedIterator 语义

所有 `lazy` 迭代器实现 `std::iter::FusedIterator`，保证：`next()` 返回 `None` 后，继续调用永远返回 `None`。实现方式：`current` 字段为 `Option<HalfEdgeId>`，一旦置 `None`，`let cur = self.current?`；立即返回 `None`。

3.5 基准测试结果

在 `icosphere`（单位球面细分网格）上对比 `lazy` 与 `eager` 的「构造 + 完整消费」总耗时与内存占用。测试环境：`cargo bench --release, std::time::Instant` 计时。

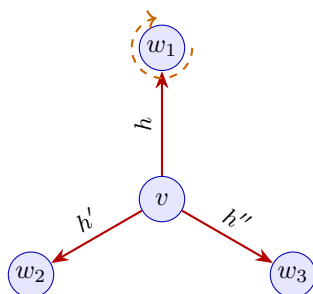
规模	顶点环 VertexRing			面边界 FaceHalfEdges			内存 (MB)	
	eager (s)	lazy (s)	比值	eager (s)	lazy (s)	比值	eager	lazy
~10K (n=5)	0.002	0.001	0.53×	0.001	0.000	0.20×	0.7 / 0.9	0
~100K (n=7)	0.016	0.008	0.53×	0.008	0.001	0.19×	11.3 / 15.0	0
~1M (n=8)	0.062	0.044	0.71×	0.026	0.007	0.25×	45.0 / 60.0	0

结论：

- **耗时**：lazy 在所有规模上均快于 eager（比值 < 1 ）。面边界遍历优势最大（ $0.19\text{--}0.25\times$ ），因为每面仅 3 条半边，eager 的 Vec 分配开销远超迭代本身。
- **内存**：lazy 零堆分配；eager 在 1M 级别分配 ~ 105 MB（顶点环 45 MB + 面边界 60 MB）。若需保留 eager 的「迭代期可 `&mut mesh`」特性又想避免重复分配，可使用 `*_into` 内存复用变体（见 §1.1）：在热循环外声明一次 `buf`，循环内 `buf.clear()` 后填充，容量自动复用。
- **一致性**：所有规模的 lazy 与 eager 产出数量完全一致（断言 $\sum d_v = |HE|$, $\sum d_f = 3|F|$ ）。

4 顶点环绕：CCW 旋转规则

设 h 是顶点 v 的某条 outgoing 半边（即 $h.\text{twin.vertex} = v$ ），则绕 v 旋转到下一条 outgoing 半边的两条等价操作：



CCW 顺序： $h \rightarrow h' \rightarrow h'' \rightarrow h$ （闭合）

4.1 两条旋转规则的推导

CCW next: $h \mapsto h.\text{prev.twin}$

- $h.\text{prev}$ 与 h 同面，按 CCW 方向排在 h 之前，结尾落在 v （因为 h 起始于 v ）；
- $h.\text{prev.twin}$ 起始于 v ，且属于「 $h.\text{prev}$ 这条边的另一侧邻接面」；
- 因此跨过 $h.\text{prev}$ 这条边相当于绕 v 旋转到逆时针方向上的相邻面，得到下一条 outgoing 半边。

CW next: $h \mapsto h.\text{twin.next}$

- $h.\text{twin}$ 是反向半边，结尾落在 v ；
- $h.\text{twin.next}$ 起始于 v ，与 h 处于同一条边界的另一侧；
- 跨过 h 这条边相当于绕 v 旋转到顺时针方向上的相邻面。

4.2 算法：单三角面片实例

设单个三角面片 $F = (h_0, h_1, h_2)$ ，其中 $h_0 : v_0 \rightarrow v_1$ ， $h_1 : v_1 \rightarrow v_2$ ， $h_2 : v_2 \rightarrow v_0$ ，每条 h_i 的 twin t_i 是边界半边（face = None，无 next/prev）。

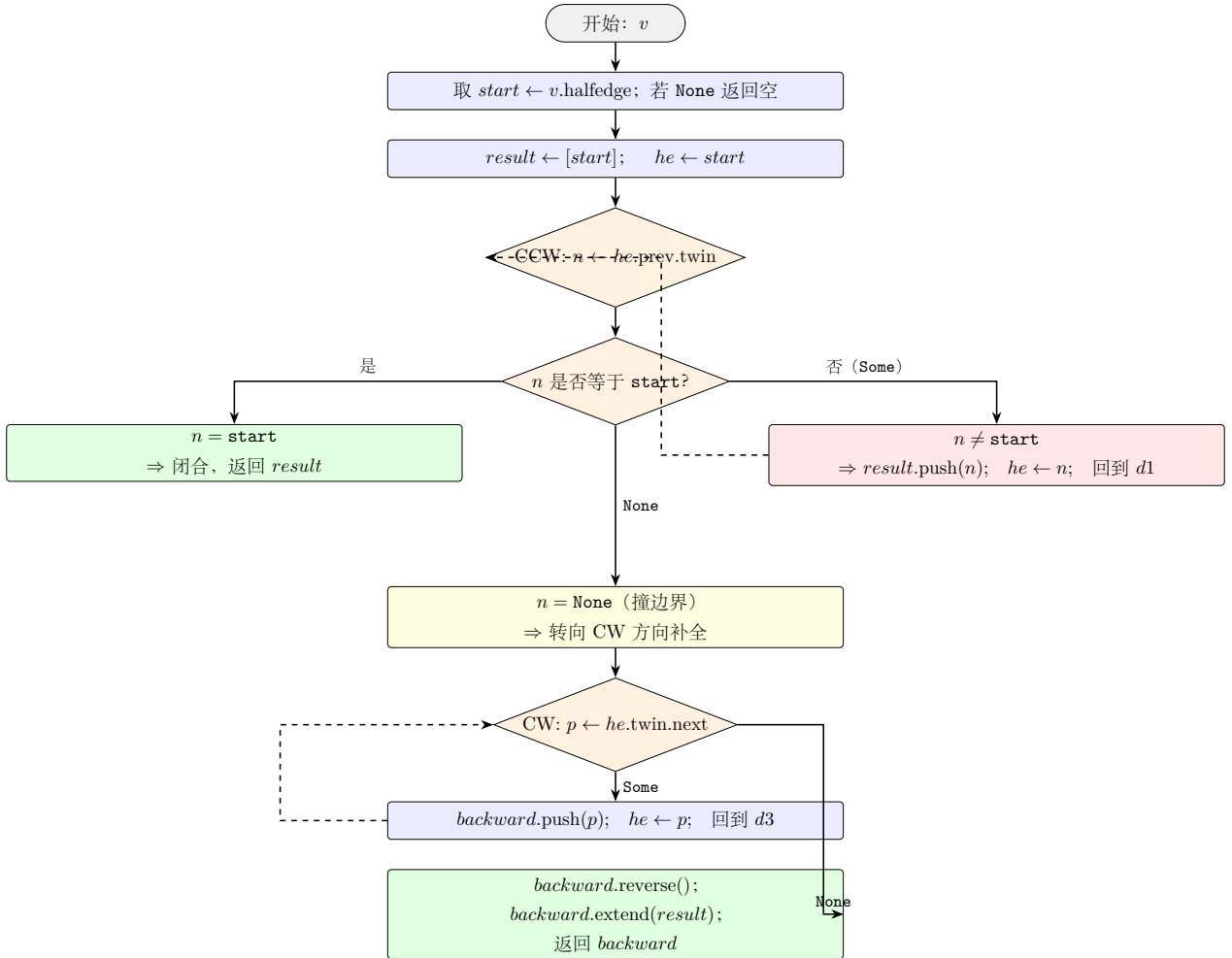
对 v_0 ， $v_0.\text{halfedge} = h_0$ ：

1. **CCW 方向**： $h_0 \rightarrow h_0.\text{prev.twin} = h_2.\text{twin} = t_2$ ；再走 $t_2.\text{prev.twin}$ ，但 $t_2.\text{prev} = \text{None}$ ，停止。
2. 未闭合 $\Rightarrow$ 是边界顶点，向 CW 方向补全： $h_0 \rightarrow h_0.\text{twin.next} = t_0.\text{next} = \text{None}$ ，立即停止。
3. 拼接： $[\].\text{reverse}() + [h_0, t_2] = [h_0, t_2]$ 。

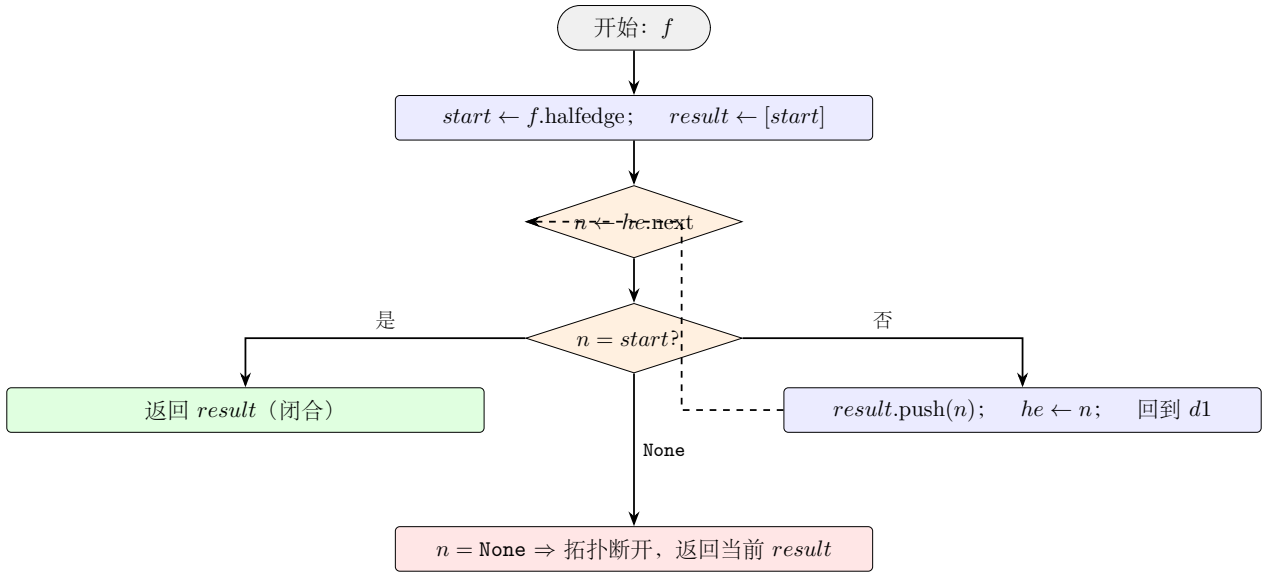
最终输出 $[h_0, t_2]$ ，CCW 角度顺序： h_0 (0°) $\rightarrow t_2$ (90°)。

5 算法流程图

5.1 顶点 outgoing 半边收集



5.2 面边界半边收集



6 边界判定

6.1 边界边

记 h 的 twin 为 h^* 。半边 h 所在的边是边界边，当且仅当：

$$\text{is_boundary_edge}(h) = (h.\text{face} = \text{None}) \vee (h^* = \text{None}) \vee (h^*.\text{face} = \text{None})$$

三种情形分别对应：「 h 自己是边界半边」「 h 没有 twin（拓扑未完整缝合）」「 h 的对侧是边界半边」。任何一种都意味着该边只有 0 或 1 个相邻面。

无效 ID 语义 若 h 已被删除或从未分配，`is_boundary_edge` 返回 `false`。**理由**：已删除的句柄不再代表任何边，因此既非内部边也非边界边。调用者若需区分「无效」与「非边界」，应先调用 `MeshStorage::contains_halfedge`：

```

1 if !mesh.contains_halfedge(he) {
2     // he " "
3 } else if is_boundary_edge(&mesh, he) {
4     // he
5 } else {
6     // he
7 }
  
```

6.2 边界顶点

顶点 v 是边界顶点，当且仅当它关联的任一 outgoing 半边所在边为边界边：

$$\text{is_boundary_vertex}(v) = \exists h \in \text{VertexRing}(v) : \text{is_boundary_edge}(h).$$

等价表述：内部顶点的 outgoing 半边集形成一个**闭合** CCW 环；边界顶点的 outgoing 半边集形成**开链**，两端都是边界半边。

7 复杂度

设顶点 v 的度为 d_v ，面 f 的边数为 d_f （三角形为 3）：

API	构造时间	构造空间	每次 <code>next()</code>
<code>VertexRing</code>	$O(d_v)$	$O(d_v)$	$O(1)$
<code>VertexAdjacentVerts</code>	$O(d_v)$	$O(d_v)$	$O(1)$
<code>VertexAdjacentFaces</code>	$O(d_v)$	$O(d_v)$	$O(1)$
<code>FaceHalfEdges</code>	$O(d_f)$	$O(d_f)$	$O(1)$
<code>FaceVertices</code>	$O(d_f)$	$O(d_f)$	$O(1)$
<code>VertexRingLazy</code>	$O(d_v)$	$O(1)$	$O(1)$
<code>VertexAdjacentVertsLazy</code>	$O(d_v)$	$O(1)$	$O(1)$
<code>VertexAdjacentFacesLazy</code>	$O(d_v)$	$O(1)$	$O(1)$
<code>FaceHalfEdgesLazy</code>	$O(1)$	$O(1)$	$O(1)$
<code>FaceVerticesLazy</code>	$O(1)$	$O(1)$	$O(1)$
<code>is_boundary_edge</code>	$O(1)$	—	—
<code>is_boundary_vertex</code>	$O(d_v)$	—	—

8 健壮性

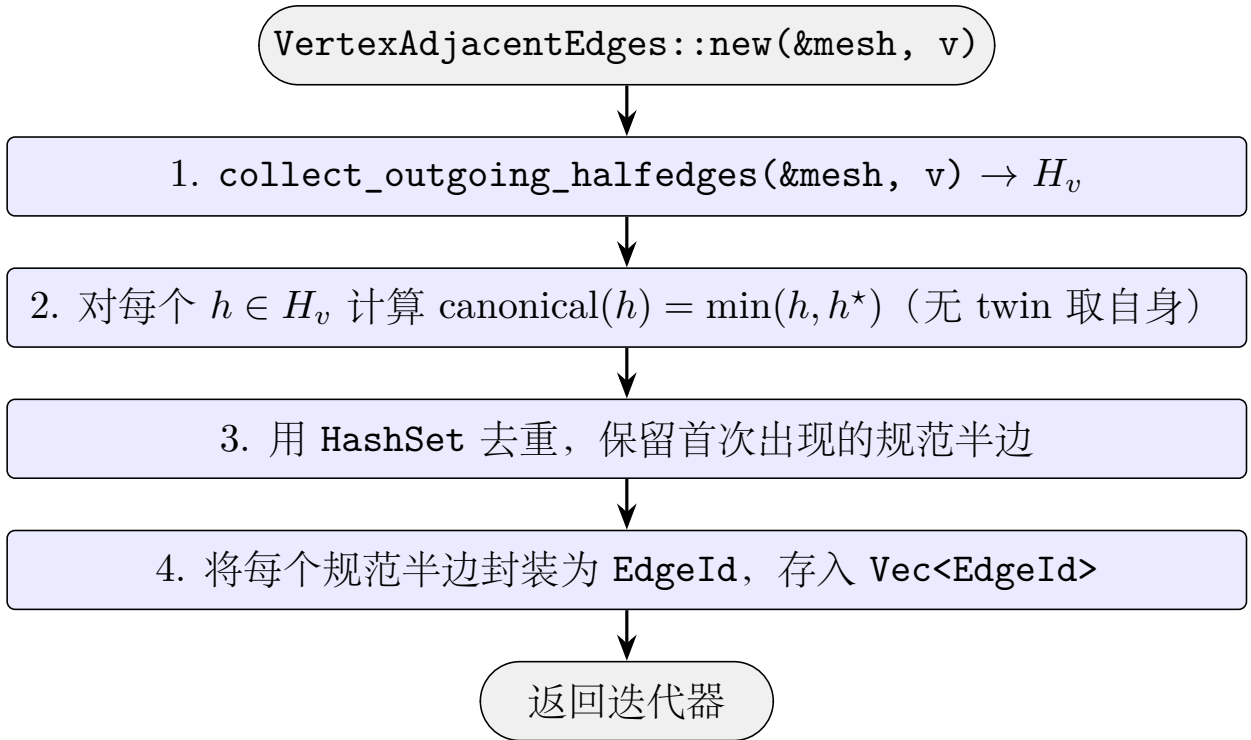
本模块对所有可能的 panic 路径都加了守卫：

- `v.halfedge` 或 `f.halfedge` 为 `None` $\Rightarrow$ 返回空 `Vec` (eager) 或空迭代器 (lazy)；
- 中途任一 `get_halfedge` 返回 `None`（半边被并发删除等） $\Rightarrow$ `and_then` 链路平滑返回 `None`，循环停止；
- 主循环上界设为 `mesh.halfedge_count() + 1`，对不一致拓扑（如自指环）兜底，避免死循环；
- lazy 迭代器的 `current` 字段为 `Option`，撞到边界或回到起点后置 `None`，后续调用 `next()` 立即返回 `None`（`FusedIterator` 语义）；
- `is_boundary_edge` 对无效 `he` 返回 `false`（非 panic）；
- `is_boundary_vertex` 对无效 / 孤立 v 返回 `false`。

9 顶点邻接无向边：VertexAdjacentEdges

`VertexAdjacentEdges` 是预收集迭代器，产出与顶点 v 关联的所有 **唯一无向边**（`EdgeId`）。

9.1 构造与算法



9.2 与 `EdgeIter` 的关系

- `EdgeIter` 枚举整张网格的所有无向边；
- `VertexAdjacentEdges` 只枚举与指定顶点 v 关联的无向边（即 v 的 outgoing 半边所在边）；
- 两者共用相同的「规范半边」约定：twin 对中 key 较小者，边界半边取自身；
- 对闭合网格, `VertexAdjacentEdges` 的产出数等于顶点度数 (`VertexRing::new(&mesh, v).count()`)。

9.3 复杂度

构造期 $O(d(v))$ ，其中 $d(v)$ 是顶点 v 的度数；迭代期 $O(1)$ per step。实现 `ExactSizeIterator`，`size_hint` 返回精确剩余数。

10 统计信息与诊断

10.1 迭代器统计接口

为方便调试与性能分析, 所有 14 个迭代器 (5 个预收集 + 5 个 lazy + `EdgeIter` + `VertexAdjacentEdges` + `BoundaryLoop` + `BoundaryLoopLazy`) 统一提供:

方法	预收集版本	lazy 版本
<code>count_hint(&self) -> Option<usize></code>	<code>Some()</code>	<code>None</code>
<code>is_empty(&self) -> bool</code>	<code>cursor >= vec.len()</code>	<code>current.is_none()</code>
<code>ExactSizeIterator</code>	✓ (提供 <code>len()</code>)	×

设计要点:

- `count_hint` 是 inherent 方法, 不消费迭代器 (`count()` 会消费 `self`);
- 预收集版本额外实现 `ExactSizeIterator`, `size_hint` 精确返回 `(n, Some(n))` (n 为剩余数);
- lazy 版本的 `is_empty` 仅检查 `current` 是否 `None`, 不预探测长度, 保持 $O(1)$ 。

10.2 MeshStorage 诊断方法

`MeshStorage` 提供 3 个拓扑诊断方法, 用于评估网格规模与迭代器内存开销:

方法	公式	复杂度
<code>max_vertex_valence()</code>	$\max_{v \in V} d_v$	$O(V \cdot \bar{d}) = O(E)$
<code>euler_characteristic()</code>	$\chi = V - E + F$	$O(1)$
<code>genus()</code>	$g = (2 - \chi)/2$	$O(1)$

说明:

- $E = |\text{HE}|/2$ (假设每条边有 twin, 流形网格成立);
- 闭合球面 $\chi = 2$, 亏格 g 的闭合曲面 $\chi = 2 - 2g$;
- 带边界曲面的亏格公式更复杂, `genus()` 按闭合曲面公式计算, 仅作参考;
- `max_vertex_valence` 用于评估预收集迭代器内存: 总量 $\approx V \times (\bar{d} \times 8 + 24)$ 字节。

10.3 基准测试: `count_hint` vs `count`

在 1M 顶点 icosphere 上对比 `count_hint` 与 `count` 的开销 (两者都包含构造 `VertexRing` 的 $O(d)$ 时间):

操作	1M 耗时	复杂度
eager <code>count_hint</code>	0.066s	$O(d)$ 构造 + $O(1)$ 读 len
eager <code>count</code>	0.063s	$O(d)$ 构造 + $O(n)$ 遍历 Vec
lazy <code>count_hint</code>	0.024s	$O(d)$ 构造 + $O(1)$ 返回 None

分析: `count_hint` 与 `count` 耗时接近, 因为主要开销在 `VertexRing::new` 的构造阶段 (收集半边到 Vec), 而非 `count` 的遍历阶段。`count_hint` 的核心价值不在速度, 而在:

1. 不消费迭代器——可在循环中先检查长度再决定是否消费；
2. 明确长度已知性——lazy 版本返回 None，调用者不会误判；
3. 语义清晰——count() 的默认实现会消费整个迭代器，对 lazy 版本是 $O(n)$ ，而 count_hint 始终 $O(1)$ 。

11 单元测试覆盖

src/traversal.rs 末尾共 62 个测试，分六组场景：

测试名	覆盖点
场景一：单个三角面片（覆盖基本顺序与边界）	
face_halfedges_in_ccw_order	面 next 顺序 $[h_0, h_1, h_2]$
face_vertices_in_ccw_order	面顶点 $[v_1, v_2, v_0]$
vertex_ring_around_v0	v_0 outgoing = $[h_0, t_2]$
vertex_ring_around_v1	v_1 outgoing = $[h_1, t_0]$
vertex_adjacent_verts_around_v0	v_0 邻居 = $[v_1, v_2]$
vertex_adjacent_faces_around_v0	v_0 邻接面 = $[F]$
boundary_detection_for_single_triangle	全部 6 条半边都是边界边
场景二：两三角形拼成的四边形（覆盖共享边与开链 v_0 ）	
two_triangle_corner_vertices_are_still_boundary	4 个顶点都是边界顶点
two_triangle_shared_vertex_has_three_outgoing	v_0 度为 3
two_triangle_v0_has_two_adjacent_faces	v_0 邻接面 = $[F_1, F_2]$
shared_edge_is_not_boundary	共享边非边界
场景三：3 三角形闭合扇形（覆盖内部顶点 / 闭合环）	
closed_fan_center_is_interior	中心 c 非边界
closed_fan_outer_vertices_are_boundary	外圈 3 顶点都是边界
closed_fan_center_ring_closes_with_three_outgoing	c outgoing 数 = 3，闭合
closed_fan_center_adjacent_faces_yields_three	c 邻接 3 个面
场景四：lazy 迭代器与预收集版本一致性	
lazy_vertex_ring_matches_eager_on_single_triangle	单三角面片 lazy = eager
lazy_vertex_ring_matches_eager_on_closed_fan	闭合扇形 lazy = eager
lazy_vertex_ring_matches_eager_on_two_triangles	两三角形开链 lazy = eager
lazy_face_halfedges_matches_eager	面边界 lazy = eager
lazy_face_vertices_matches_eager	面顶点 lazy = eager
lazy_adjacent_verts_matches_eager	邻接顶点 lazy = eager
lazy_adjacent_faces_matches_eager	邻接面 lazy = eager
lazy_iterators_handle_invalid_ids	无效 / 孤立 ID 返回空
lazy_iterators_are_fused	next() 返回 None 后继续 fused
lazy_iterators_on_icosphere	icosphere(1) 全顶点 lazy = eager
场景五：统计信息与诊断方法	
eager_count_hint_returns_some_remaining	预收集 count_hint 返回精确剩余数
eager_is_empty_on_invalid_ids	无效 ID 的 is_empty 返回 true
eager_exact_size_iterator_size_hint	size_hint 返回精确 (lower, Some(upper))
lazy_count_hint_returns_none	lazy count_hint 恒返回 None
lazy_is_empty_on_valid_and_invalid	lazy is_empty 正确判定

测试名	覆盖点
lazy_is_empty_after_exhaustion	耗尽后 <code>is_empty</code> 为 <code>true</code>
max_vertex_valence_single_triangle	单三角形最大度数 = 2
max_vertex_valence_closed_fan	闭合扇形最大度数 = 3
max_vertex_valence_empty_mesh	空网格返回 0
max_vertex_valence_isolated_vertices	孤立顶点返回 0
max_vertex_valence_icosphere	<code>icosphere(1)</code> 度数在 [5,6]
euler_characteristic_single_triangle	$\chi = 1, g = 0$
euler_characteristic_closed_fan	$\chi = 1$
euler_characteristic_icosphere_is_two	球面 $\chi = 2, g = 0$
euler_characteristic_empty_mesh	空网格 $\chi = 0$ 不 panic
通用：设计约束与无效输入	
iterator_does_not_hold_borrow	迭代器存活期可同时 <code>&mut mesh</code>
can_modify_mesh_during_iteration	收集后可基于 ID 修改网格
invalid_or_isolated_ids_yield_empty	孤立 / 已删除 ID 安全返回空或 <code>false</code>
场景六：无向边迭代器、闭合判定与边界环	
edge_count_single_triangle	单三角形 <code>edge_count</code> =3
edge_count_two_triangles	两三角形共享边 <code>edge_count</code> =5
edge_count_closed_fan	闭合扇形 <code>edge_count</code> =6
edge_count_empty	空网格 <code>edge_count</code> =0
edge_ids_single_triangle_yields_three_unique_edges	3 条无向边互不相同
edge_ids_two_triangles_yields_five_unique_edges	5 条无向边
edge_ids_on_icosphere	<code>icosphere(1)</code> 120 条边
edge_id_vertices_on_icosphere	边的两端点查询
edge_iter_is_exact_size	<code>EdgeIter</code> 实现 <code>ExactSizeIterator</code>
edge_count_matches_euler	欧拉公式验证 <code>edge_count</code>
is_closed_single_triangle_is_false	单三角形非闭合
is_closed_closed_fan_is_false	扇形非闭合
is_closed_icosphere_is_true	<code>icosphere</code> 闭合
boundary_loop_single_triangle	单三角形 1 个边界环
boundary_loop_icosphere_is_empty	闭合网格无边界环
boundary_loop_lazy_matches_eager	<code>lazy</code> 与 <code>eager</code> 一致
boundary_loop_detects_multiple_loops	多边界环检测
场景七：顶点邻接无向边 (<i>VertexAdjacentEdges</i>)	
vertex_adjacent_edges_triangle	单三角形 v_0 关联 2 条无向边
vertex_adjacent_edges_icosphere	<code>icosphere(1)</code> 关联边数 = 度数
场景八：内存复用变体 (<i>*_into</i>)	
collect_outgoing_halfedges_into_matches_original	<code>_into</code> 产出与原函数一致 (<code>icosphere</code> 全顶点)
collect_face_halfedges_into_matches_original	<code>_into</code> 产出与原函数一致 (<code>icosphere</code> 全面)
collect_outgoing_halfedges_into_reuses_capacity	<code>buf</code> 容量跨调用不收缩
collect_face_halfedges_into_clears_buffer	预填垃圾数据后被 <code>clear</code> ，长度 = 3
collect_outgoing_halfedges_into_isolated_vertex	孤立顶点 <code>buf</code> 保持空

cargo test 全库共 503 个用例，全部通过 (0 失败 / 0 警告)。

12 设计取舍

- **为何不实现 `DoubleEndedIterator`?** 双向迭代要求保存可逆状态，但本实现的核心目标是「不持有借用」与「遍历期可修改网格」，双向能力会带来额外 API 表面积而收益有限。保持极简。
- **为何不在构造期直接 `yield`?** Rust 的 `Iterator` trait 要求 `next()` 返回 `Option`，但不能在迭代期间持有 `&mesh`。把收集与迭代解耦是同时满足「不持有借用」与「符合 `Iterator` trait」的唯一简洁方案。
- **为何用 `halfedge_count()` 作为循环上界?** 在流形一致网格中，正确的旋转环长度严格不超过半边总数。一旦拓扑不一致，该上界保证算法必然终止，而不会进入死循环——这是**健壮性**与**性能**的零代价折中。
- **为何 `lazy` 与 `eager` 并存?** 两者面向不同场景：`eager` 适合迭代期需 `&mut mesh` 或随机访问；`lazy` 适合大规模只读遍历（零堆分配，基准测试快 0.19–0.71×）。`::lazy` 构造函数返回独立的 `Lazy` 类型，调用方按需选择，无隐式开销。
- **何时使用 `*_into` 内存复用变体?** 当满足全部以下条件时优选：(1) 热循环中反复遍历不同顶点 / 面；(2) 需要随机访问或迭代期 `&mut mesh`（故不能用 `lazy`）；(3) 单次遍历结果用完即弃。此时在循环外声明一次 `buf`，循环内调用 `collect..._into(&mesh, id, &mut buf)`，`buf` 容量自动复用，避免每轮重新分配。若仅需纯只读遍历，`lazy` 仍是更优选择（零堆分配）。
- **为何 `lazy` 顶点环需要 `CW` 探测?** 开链顶点的 `outgoing` 环不是闭合的，不能像面边界那样「回到起点即终止」。CW 探测找到开链尽头后，从尽头沿 CCW 迭代即可覆盖整条链，且产出顺序与预收集版本完全一致。
- **为何 `count_hint` 返回 `Option` 而非 `usize`?** 预收集版本知道精确长度 (`Some`)，`lazy` 版本长度未知 (`None`)。返回 `Option` 让调用者明确区分两种情况，避免对 `lazy` 迭代器误用 `len()` 导致运行时错误。
- **为何 `max_vertex_valence` 放在 `traversal.rs` 而非 `storage.rs`?** 该方法依赖 `VertexRing` 遍历顶点 `outgoing` 环，而 `VertexRing` 定义在 `traversal.rs` 中。Rust 允许在多个模块对同一类型 `impl` 方法，故将依赖遍历的方法放在遍历模块，保持 `storage.rs` 的存储层纯粹性。`euler_characteristic` 和 `genus` 仅需计数，放在 `storage.rs` 中。